Robotics and Flexible Automation

Lecture 5
Trajectory planning

Prof. Zengxi (Stephen) Pan

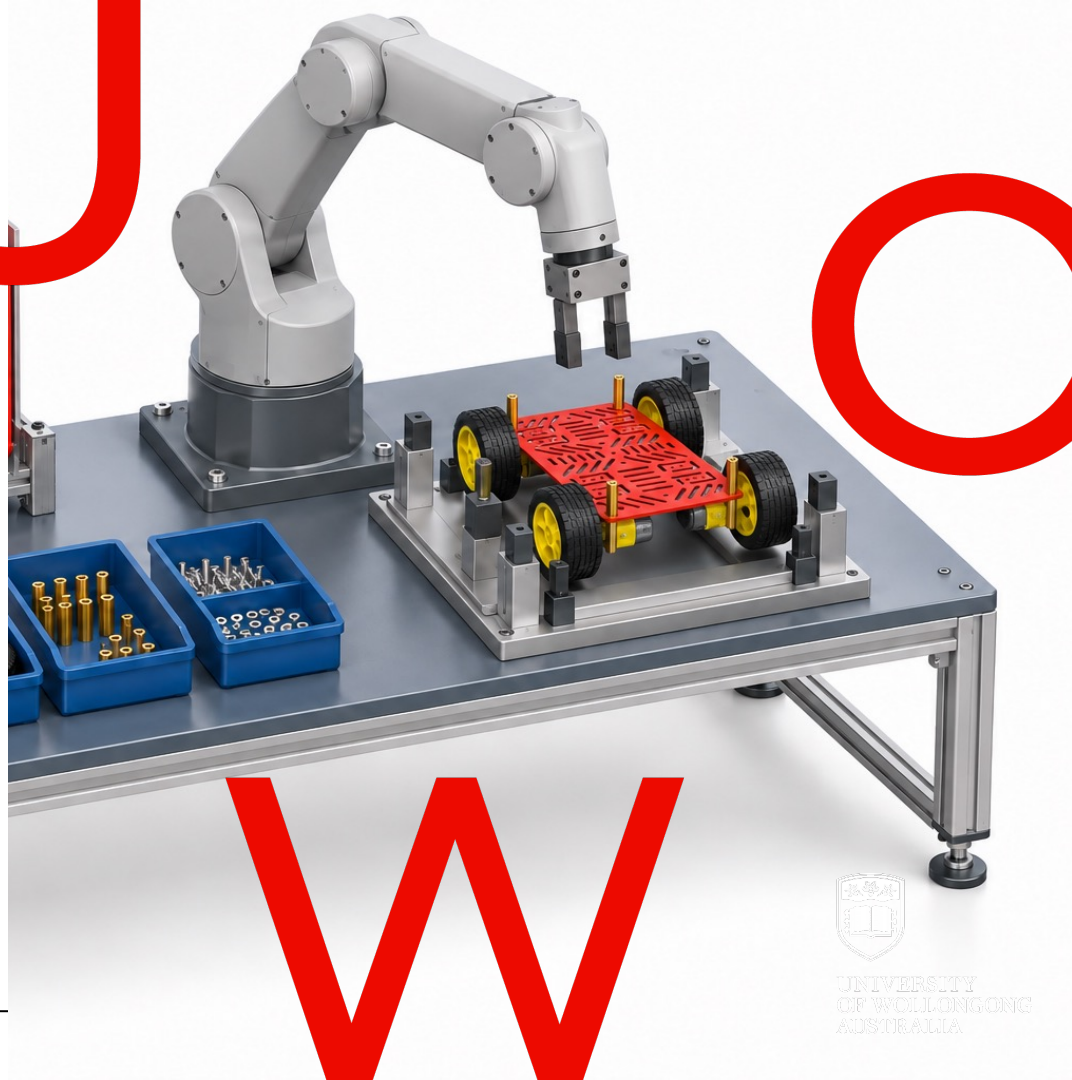


Table of Contents

CORE

1 Trajectory planning foundations

2 Joint and Cartesian trajectories

3 Toolbox methods for trajectory generation

4 Verification and application

PART 1

Trajectory planning foundations

What the robot should do before a time law is selected

1. Path-planning context and trajectory planning
2. Joint motion and Cartesian linear motion
3. Workflow targets, zones and via points

What You Will Learn

CORE

Generate and interpret trajectories with Peter Corke toolboxes

CONCEPTS

Path-planning context and trajectory
Complete planning workflow
Joint-space and Cartesian-space motion
Boundary conditions and smoothness

TOOLBOX SKILLS

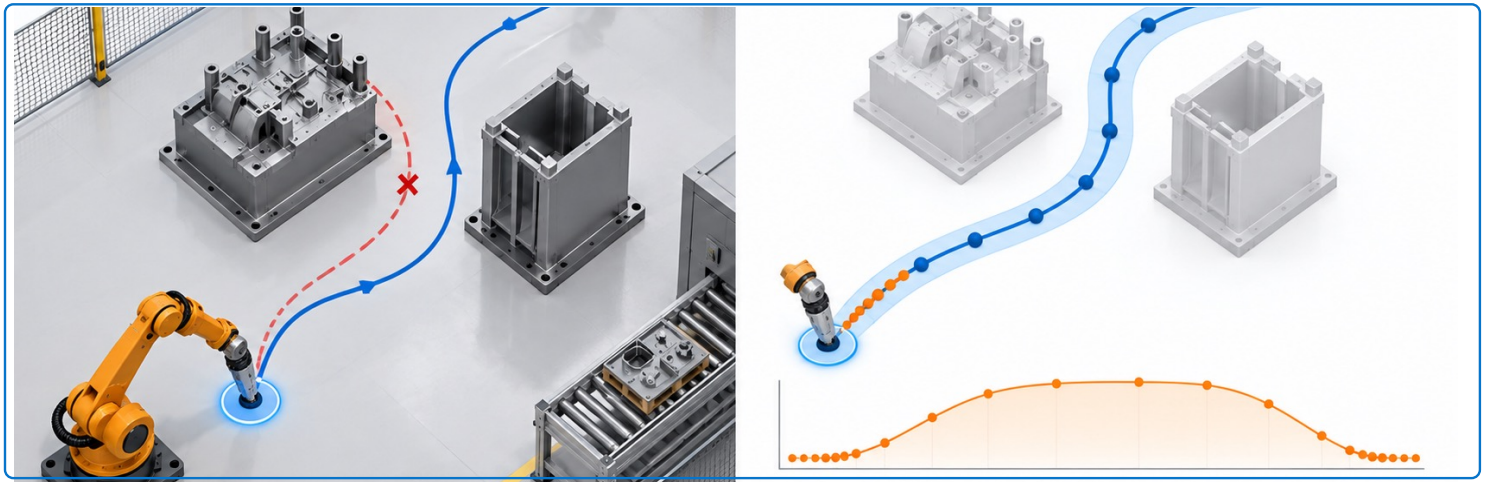
tpoly, lspb
mtraj, jtraj, mstraj
trinterp, ctraj and orientation interpolation

By the end, you should be able to generate the trajectory, plot it and explain what each function controls.

Path Planning: Context for Trajectory Planning

PATH PLANNING

TRAJECTORY PLANNING



Start, goal and obstacles define a collision-free geometric route. Timing is not assigned yet.

A time law assigns position, velocity and acceleration along the chosen path.

Path planning answers WHERE. Trajectory planning answers WHEN and HOW FAST.

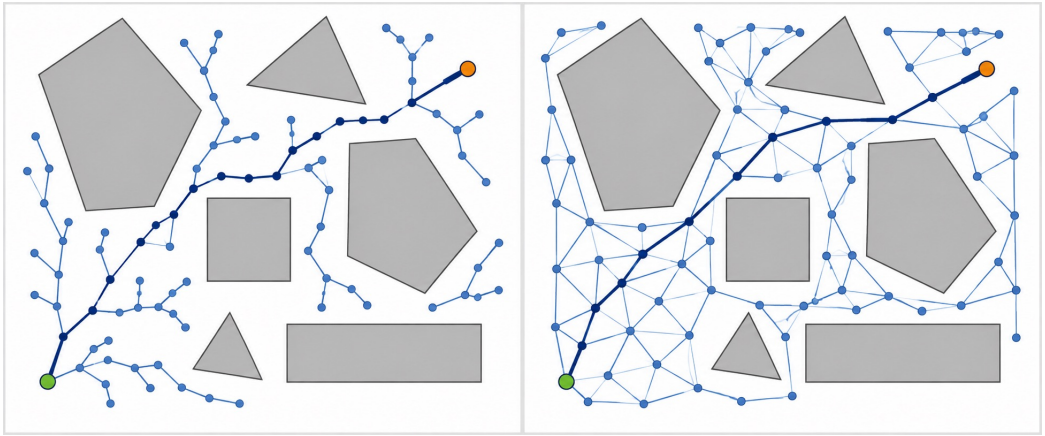
Path Planning: Brief Preview

OPTIONAL

RRT and PRM are introduced here; Lecture 9 covers path planning in detail

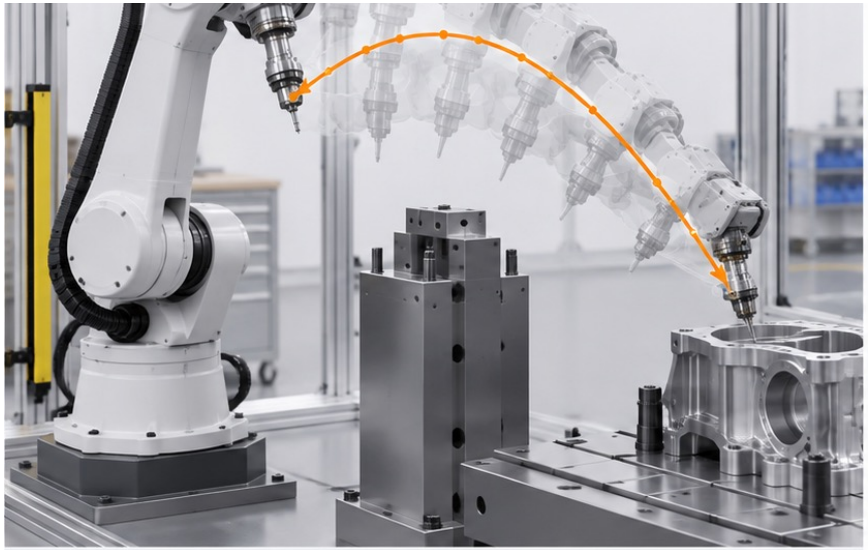
RRT
Samples a new state, extends the nearest branch, and searches for one start-to-goal route. It suits high-dimensional single-query problems.

PRM
Samples collision-free milestones, connects nearby valid motions, then searches the roadmap. The same roadmap can answer many queries.

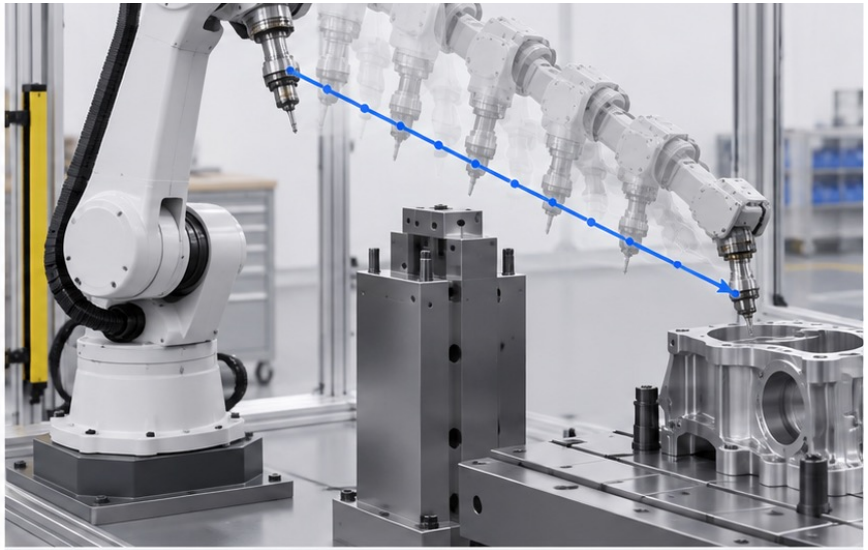


PRM and RRT are two foundational design patterns of sampling-based planning.

Joint and Linear Motion



JOINT/POINT-TO-POINT MOVE
q = jtraj(q0,q1,t)
Direct joint-coordinate interpolation
TCP path is not prescribed



LINEAR MOVE
Ts = ctraj(T0,T1,numel(t))
Straight TCP translation
IK converts poses to joints

A Motion Plan for Panel Installation

EXAMPLE

Run CAD simulation from the course website

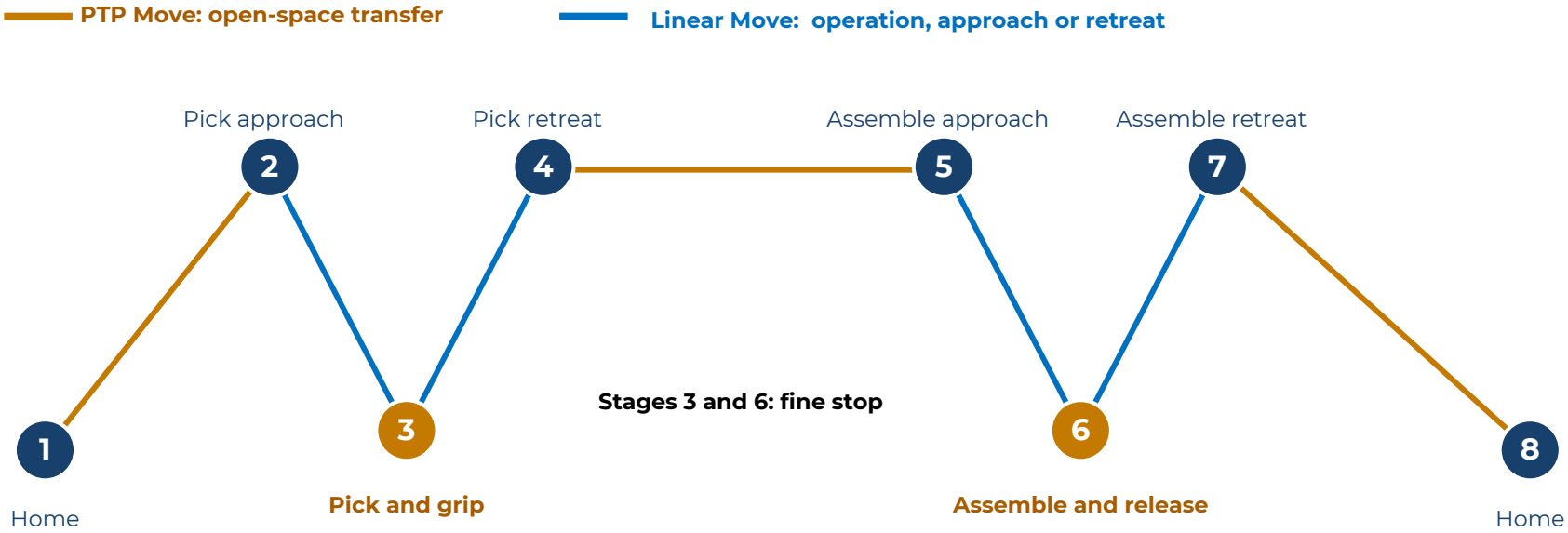
Panel installation
Ten-part pick-and-place sequence

Open simulation on the course website

Pick-and-Place Workflow

CORE

Eight stages from home to pickup, assembly and return



PTP Move handles open-space transfers.
Linear Move controls approach and retreat.
Stages 3 and 6 use fine stops for grip and release.

Targets, Fine Stops and Zones in the Workflow

CORE

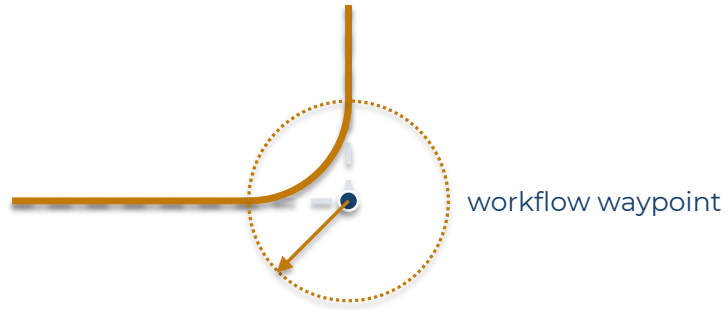
Apply stop and blend decisions to the eight-stage workflow

Fine stop

Stages 3 and 6 reach the pick and assembly poses, then settle before grip or release.

Zone / blend

Stages 2, 4, 5 and 7 may use a zone when the surrounding clearance permits blending.



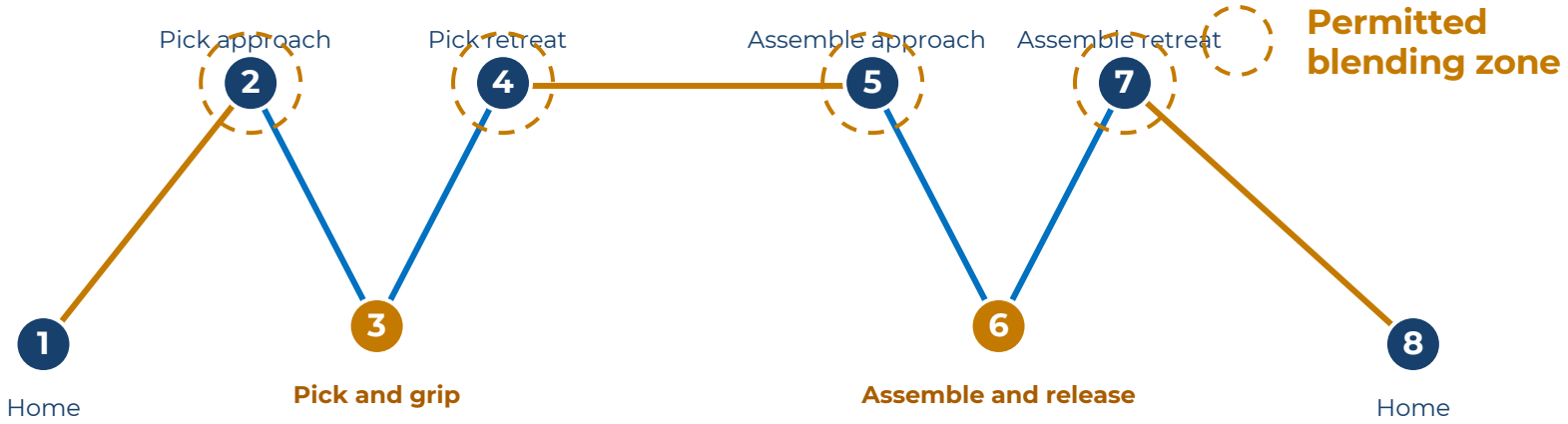
fine reaches the programmed point

The waypoint defines the route. The zone defines how closely the robot passes it.

Zones in the Eight-Stage Workflow

CORE

A zone defines the permitted blend around a workflow waypoint



Stages 2, 4, 5 and 7 may use zones when the surrounding path has enough clearance.

Stages 3 and 6 remain fine stops because the tool must reach the pick and assembly poses.

Summary: Motion Type and Zone Selection

CORE

Apply Point-to-Point (PTP) and Linear Moves across all seven segments

Segment	Motion	Zone
1 to 2: Home to pick approach	PTP Move	zone allowed
2 to 3: Pick approach to pick	Linear Move	fine at 3
3 to 4: Pick to pick retreat	Linear Move	zone allowed at 4
4 to 5: Pick retreat to assembly approach	PTP Move	zone allowed
5 to 6: Assembly approach to assembly	Linear Move	fine at 6
6 to 7: Assembly to assembly retreat	Linear Move	zone allowed at 7
7 to 8: Assembly retreat to home	PTP Move	fine at 8

PTP Move handles open-space transfer. Linear Move controls approach and retreat direction. Fine stops belong at grip and release.

Additional Via Points for Collision Avoidance

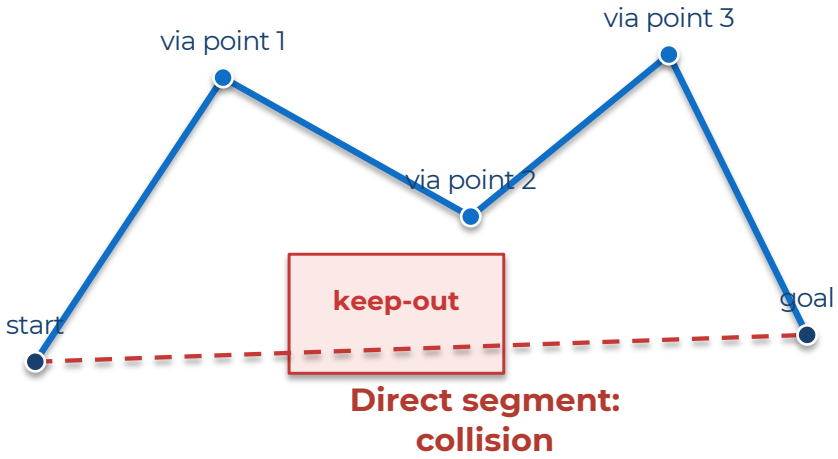
CORE

Add via points when a direct segment intersects an obstacle or violates clearance

Start with the minimum path: only the required workflow targets.

Check every segment for collision and clearance.

If a direct segment fails, add the fewest via points needed to create a feasible route.



Minimum workflow

Use only the required process and approach/retreat targets when the path is clear.

Collision requirement

Add via points only where geometry requires a detour, then recheck the complete path.

PART 2

Joint and Cartesian trajectories

Generate joint references directly or plan the TCP route then solve IK

1 Constraints and joint-space trajectories
2 Cartesian trajectories and continuous IK
3 Executed joint references

Constraints on Trajectory Generation

CORE

A valid trajectory satisfies boundary, kinematic, geometric and process constraints

Boundary

Start and goal poses, via points, and the velocity at each end of a segment.

Kinematic

Joint position, velocity and acceleration limits; jerk where the controller exposes it.

Geometry

Collision, clearance, reach and the orientation the process demands.

Process

Contact force, settling, quality and the cycle time the client bought.

Every one of these must hold continuously along the motion — satisfying them at the sampled targets proves nothing in between.

Joint-Space Trajectory

CORE

Interpolate the six joint coordinates directly

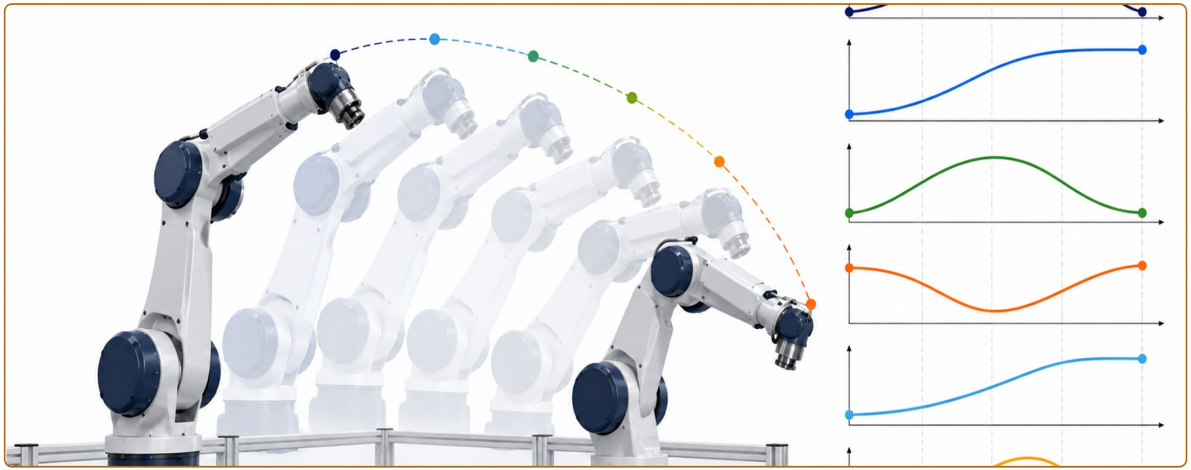
q(t) = qA + s(t)(qB - qA)

Input: qA, qB, duration and joint limits

Generate one smooth qi(t) for each axis

Synchronise all axes to arrive together

The TCP path follows from forward kinematics



No inverse kinematics is required along the motion. Use this for efficient free-space transfer.

Joint-Space Planning

CORE

Simple and robust, but it does not constrain the TCP curve

Advantages

No inverse kinematics at the interpolated points.

Joint limits are expressed directly.

No Cartesian singularity can stop the motion.

Limitations

The TCP route follows from the kinematics and is hard to predict.

Clearance must be checked over the whole swept volume.

Process direction is not prescribed.

The right default for free-space transfer — once the swept volume has been checked in simulation.

Cartesian-Space Trajectory

CORE

Prescribe the TCP path first, then calculate the joint motion

$$T(t) = TA + s(t) \times (TB - TA)$$

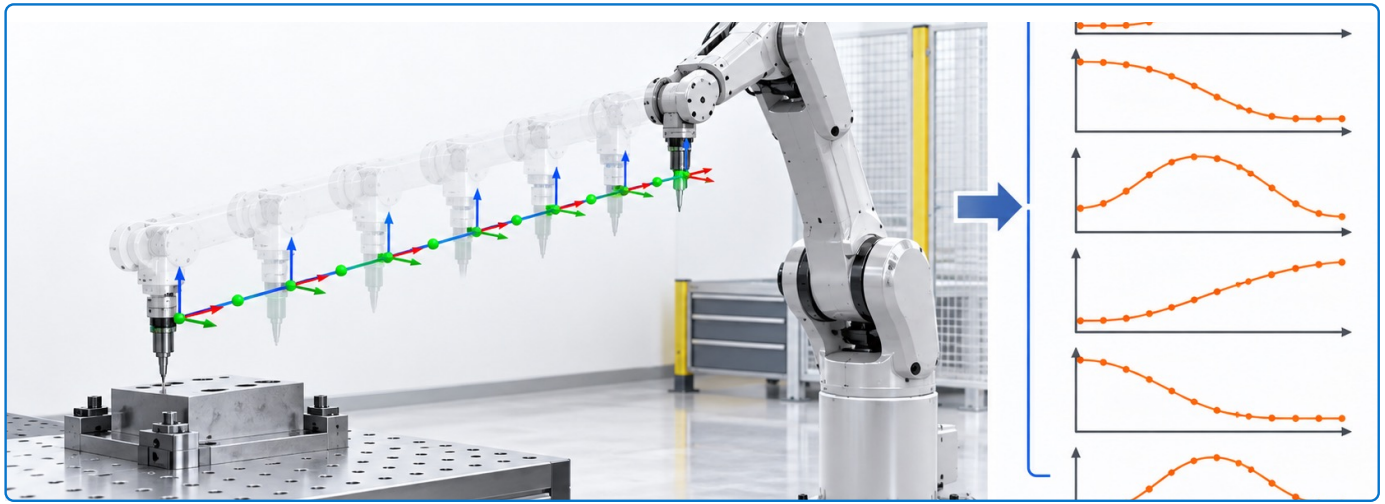
$$T(t) \rightarrow IK \rightarrow q(t)$$

1 Interpolate the TCP pose along a line

2 Apply the time law s(t)

3 Solve IK at every sampled pose

4 Keep one continuous branch and check limits, singularities and collision

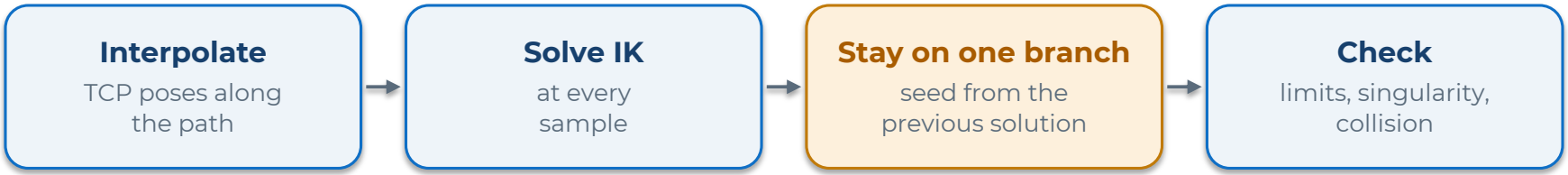


But we can't really do (TB - TA) , the orientation needs to be properly interpolated.

Cartesian Planning Requires IK Along the Path

CORE

A Cartesian route must become a continuous, feasible joint trajectory



Sampling too coarsely hides exactly the configurations that cause trouble — refine near contact and near the reach boundary.

A configuration change mid-path is a large, fast joint motion even though the TCP barely moves; most controllers refuse it inside a linear move.

Seed each IK solve with the previous joint solution. It is one line of code and it is what keeps a Cartesian path on one branch.

PART 3

Toolbox methods for trajectory generation

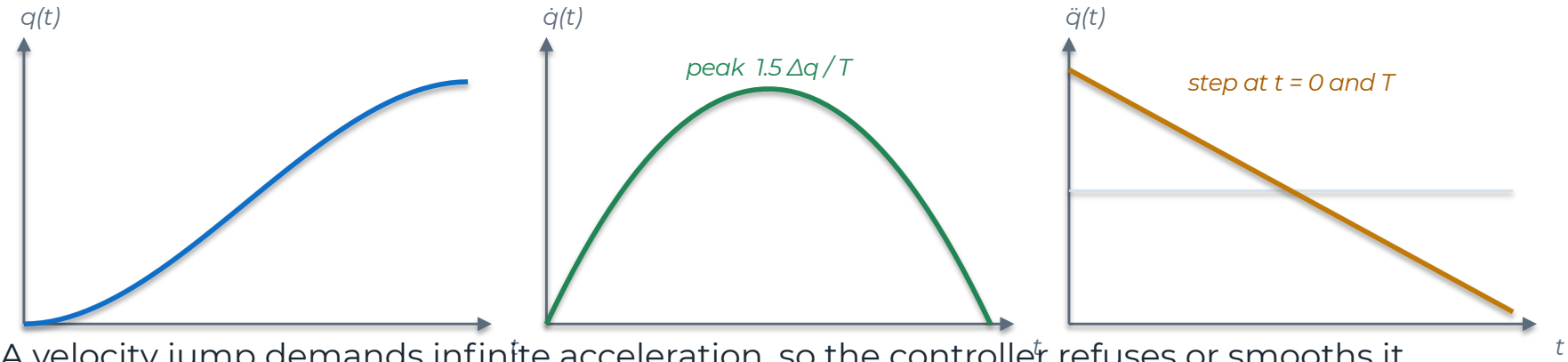
Generate, interpolate and verify trajectories with the toolbox

1. Time laws and joint-space functions
2. Cartesian and orientation interpolation
3. MATLAB examples and motion checks

Smoothness: Position, Velocity, Acceleration

CORE

A usable trajectory is continuous in q and $\dot{q}$, and bounded in $\ddot{q}$



A velocity jump demands infinite acceleration, so the controller refuses or smooths it.

An acceleration step is finite but excites vibration — it shows up as marking or overshoot.

Continuity is not cosmetic. It is what keeps joint torque, settling time and process quality inside their limits.

Example: Cubic Polynomial Time Law

WORKED EXAMPLE

Four boundary conditions, four coefficients

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

$$q(0) = q_0,\ q(T) = q_1,\ \dot{q}(0) = \dot{q}(T) = 0$$

$$a_0 = q_0, \quad a_1 = 0, \quad a_2 = \frac{3\,\Delta q}{T^2}, \quad a_3 = -\frac{2\,\Delta q}{T^3}$$

Quantity	Peak value
speed |q̇|	1.5 Δq / T at t = T/2
acceleration |q̈|	6 Δq / T² at t = 0 and t = T

Size T from the peak speed, then check the acceleration.

Mean speed is 2/3 of peak; acceleration and deceleration use cycle time.

These scalar profiles assume $\Delta q = q_1 - q_0 > 0$. For negative moves, use $|\Delta q|$ for peak magnitudes; retain the sign in q(t).

Quintic Time Law

OPTIONAL

Choose tpoly when smooth start/finish behaviour matters more than a constant-speed cruise

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \qquad \tau = t/T$$

Adds $\ddot{q}(0) = \ddot{q}(T) = 0$, so the acceleration is continuous at the ends of the segment as well as the velocity.

Peak speed is 1.875 $\Delta q/T$. Peak acceleration is 5.77 $\Delta q/T^2$, slightly below the cubic value of 6 $\Delta q/T^2$.

This is what the toolbox tpoly and jtraj functions generate by default.

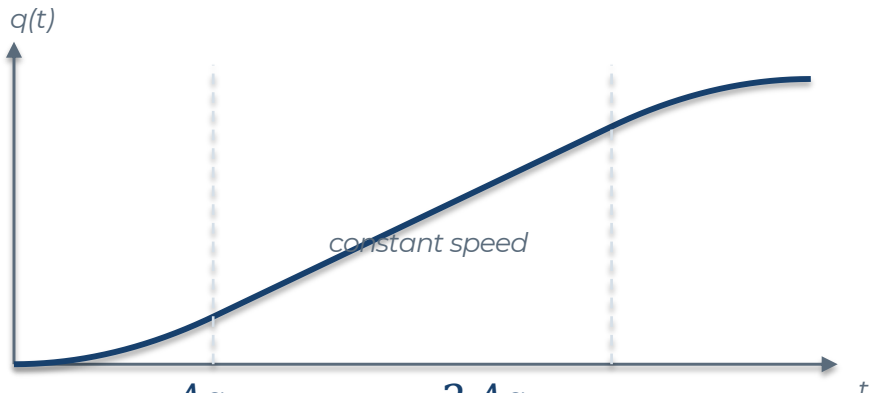
Assume $\Delta q > 0$ here; for negative moves, use $|\Delta q|$ in peak magnitudes. tpoly and jtraj retain the motion direction.

Trapezoidal Velocity Profile

CORE

Choose lspb when a controlled constant-speed middle phase is useful





$$\Delta q = v_{max}\,(T - t_a), \qquad a = \frac{v_{max}}{t_a}, \qquad \frac{\Delta q}{T} < v_{max} \leq \frac{2\,\Delta q}{T}$$

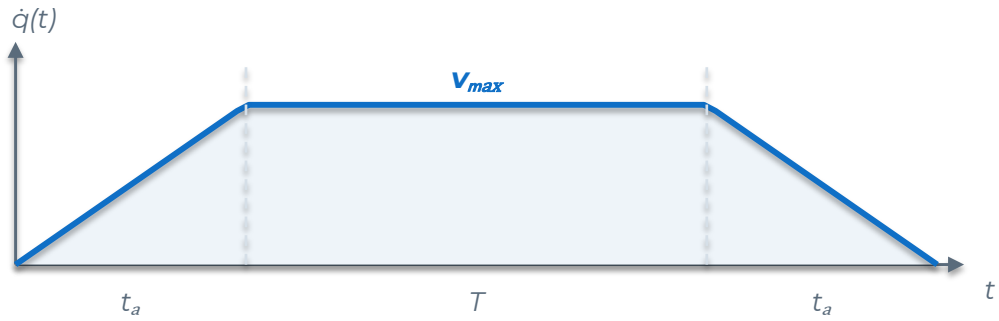
Assume Δq > 0. At v = 2Δq/T the profile is triangular. For reverse motion, use |Δq| in these speed bounds.

Choosing tpoly or lspb

CORE

Choose the time law that matches the motion, then apply it to joints or poses

```
[sp, vp, ap] = tpoly(0,1,50);
[sl, vl, al] = lspb(0,1,50);
plot([vp vl]); legend('tpoly','lspb');
mean(vp)/max(vp)     % 0.523102
mean(vl)/max(vl)     % 0.653061
```



tpoly gives smooth rest-to-rest endpoint behaviour.

lspb includes a constant-speed cruise.

For 50 samples, mean/peak speed is about 0.523 and 0.653 respectively. The continuous default LSPB ratio is 2/3; sampling changes the average.

Always compare peak speed and acceleration with axis limits before execution.

Example: Point-to-Point Move Sizing

WORKED EXAMPLE

Axis 1 must turn 90°, limited to 60°/s and 120°/s2

Step	Calculation	Result
Blend time	t_a = v_max / a_max = 60 / 120	0.50 s
Travel in the two blends	v_max · t_a = 60 × 0.50	30°
Cruise travel	90 − 30	60°
Cruise time	60 / 60	1.00 s
Segment time	T = 1.00 + 2 × 0.50	2.00 s

check: $\Delta q = v_{max}(T - t_a) = 60\,(2.00 - 0.50) = 90^\circ$ ✓

Always substitute back. A time law that does not reproduce the commanded travel is a sign error, not a rounding error.

Example: Joint and Multi-Segment Trajectories

WORKED EXAMPLE

jtraj for one segment, mstraj for a sequence through via points

```
q0 = zeros(1,6); q1 = [0 .5 -.5 0 .5 0];
t = (0:49)'*0.02;   % seconds
[q,qd,qdd] = jtraj(q0,q1,t);
via = [4 1;4 4;5 2;2 5];
qdmax = [2 1];
qs = mstraj(via,qdmax,[],[4 1],.05,0);
qb = mstraj(via,qdmax,[],[4 1],.05,1);
% qs: sharp corner; qb: blended corner
```

jtraj uses a shared time vector for all joints.

mstraj uses per-axis speed limits and a blend time.

Zero blend gives sharp velocity changes, not stops. A nonzero blend rounds corners. Use separate rest-to-rest segments when a stop is required.

These are planning tools, not verification tools. A plotted trajectory is not a collision-checked cell motion.

Example: Cartesian Trajectories

WORKED EXAMPLE

Interpolate poses, solve IK continuously, then verify

```
mdl_puma560; rob=p560;
T0=transl(.5,-.2,.35)*trotx(pi);
T1=transl(.5, .2,.35)*trotx(pi);
t=(0:49)'*.02; Ts=ctraj(T0,T1,lspb(0,1,t));
qk=rob.ikine6s(T0,'ruf');
qk=atan2(sin(qk),cos(qk));
q=zeros(numel(t),rob.n);
for k=1:numel(t)
    qr=rob.ikine(Ts(:,:,k),'q0',qk);
    assert(~isempty(qr) && all(isfinite(qr)));
    qk=qr+2*pi*round((qk-qr)/(2*pi));
    q(k,:)=qk;
end
err=transl(rob.fkine(q))-transl(Ts);
```

The Puma model uses a verified starting branch ('ruf'). Samples are 0.02 s apart.

Choose the nearest equivalent revolute angle to avoid artificial ±π jumps.

Check the full FK pose, joint limits and continuity.

ikine accepts a pose sequence directly, but the explicit loop makes the seeding visible — and lets you stop at the first failure.

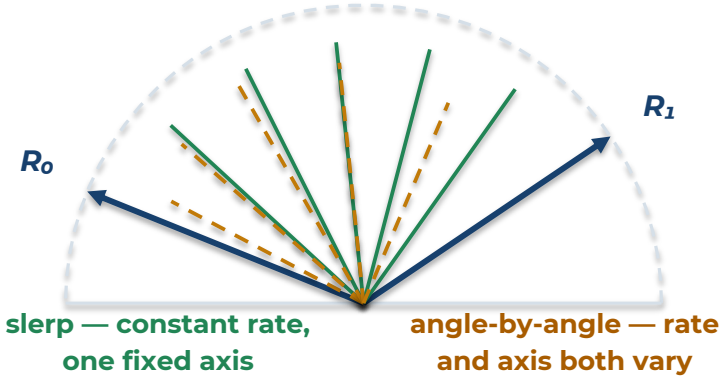
Orientation Interpolation

CORE

Position and orientation both need a continuous, physically meaningful rule

$$T(s) = \begin{bmatrix} R(s) & p(s) \\ 0 & 1 \end{bmatrix}$$

Representation	Interpolation behaviour
Euler / RPY angles	easy to read; rate and axis both vary, and it breaks down near representation singularities
Rotation matrix	element-wise blending leaves the matrix non-orthonormal — it must be re-normalised
Unit quaternion	slerp: fixed axis, constant rate for uniform s.



State the approach direction the process requires first; then choose a representation that can deliver it.

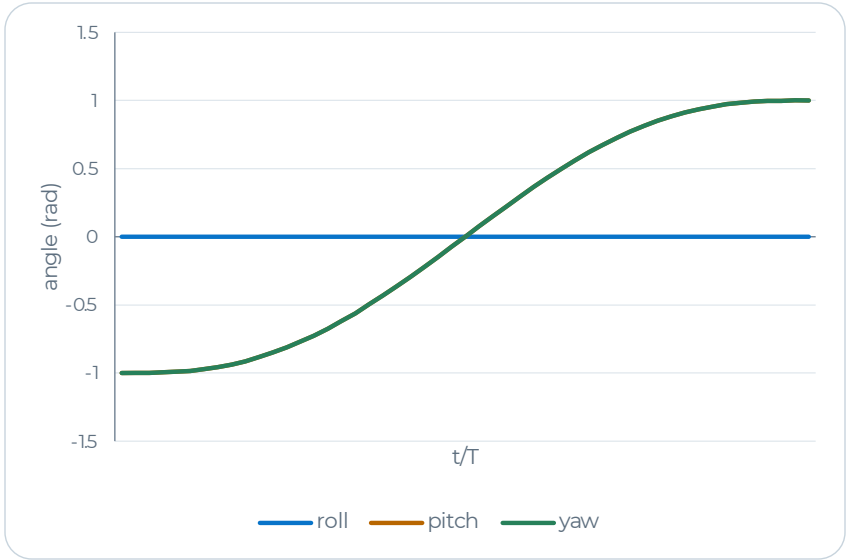
RPY Angle Interpolation

CORE

mtraj interpolates roll, pitch and yaw independently

```
R0 = rotz(-1)*roty(-1);
R1 = rotz( 1)*roty( 1);

rpy0 = tr2rpy(R0);
rpy1 = tr2rpy(R1);
rpy  = mtraj(@tpoly,rpy0,rpy1,51);
tranimate(rpy2tr(rpy));
```



Smooth angle curves do not guarantee a fixed physical rotation axis

Example: Two Ways to Interpolate Orientation

WORKED EXAMPLE

Identical endpoints, different paths; relative endpoint angle is about 146°

```
R0 = rotz(-1)*roty(-1);
R1 = rotz( 1)*roty( 1);
% angle-by-angle on an RPY triple
r0 = tr2rpy(R0); r1 = tr2rpy(R1);
rpy = mtraj(@tpoly,r0,r1,50);
tranimate(rpy2tr(rpy));
% quaternion slerp: one fixed axis
q0 = UnitQuaternion(R0);
q1 = UnitQuaternion(R1);
q = interp(q0,q1,linspace(0,1,50));
q.animate();   % accepts quaternion objects
```



Same endpoints do not imply the same intermediate tool orientation.

Animate both paths and check tool, wrist and fixture clearance.

Interpolating angles is not the same as interpolating orientation.

Complete MATLAB / Corke RTB10 Example

EXAMPLE

The same endpoints can define a joint trajectory or a Cartesian pose trajectory

```
% JOINT SPACE: metres, radians, seconds
mdl_puma560; rob=p560;
T0=transl(.5,-.2,.35)*trotx(pi);
T1=transl(.5, .2,.35)*trotx(pi);
t=(0:49)'*.02;
q0=rob.ikine6s(T0,'ruf');
q0=atan2(sin(q0),cos(q0));
q1=rob.ikine(T1,'q0',q0);
assert(~isempty(q1) && all(isfinite(q1)));
q1=q1+2*pi*round((q0-q1)/(2*pi));
[qJ,qdJ,qddJ]=jtraj(q0,q1,t);
rob.plot(qJ);
```

```
% SAME rob, T0, T1, t and q0
Ts=ctraj(T0,T1,lspb(0,1,t));
q=zeros(numel(t),rob.n); qk=q0;
for k=1:numel(t)
    qr=rob.ikine(Ts(:,:,k),'q0',qk);
    assert(~isempty(qr) && all(isfinite(qr)));
    qk=qr+2*pi*round((qk-qr)/(2*pi));
    q(k,:)=qk;
end
rob.plot(q);
```

Both examples use the same model, targets and time samples (50 points, dt = 0.02 s). IK checks pose feasibility, not collision clearance.

Example: Checking a Planned Motion

WORKED EXAMPLE

Use the same robot, trajectory and time samples as the previous example

```
% q, t and rob from slide 32
dt = t(2)-t(1); qlim = rob.qlim;
assert(numel(t) == size(q,1));
qd = diff(q)/dt; qdd = diff(qd)/dt;
max(abs(qd))       % compare with axis limits
lo = q < qlim(:,1).'; hi = q > qlim(:,2).';
assert(~any(lo | hi,'all')); % reject violations
p = transl(rob.fkine(q));
v = vecnorm(diff(p),2,2)/dt;
m = arrayfun(@(k) rob.maniplty(q(k,:)), ...
             1:size(q,1));
```

Joint speed and acceleration against the rated limits.

Joint position against the travel limits, every sample.

TCP speed, to confirm the process speed promised.

Manipulability, to show where the path nears a singularity.

Collision is the one item this script cannot check — that needs a geometric model of the cell, tool and carried part.

Trajectory Planning Exercise

EXAMPLE

Generate the trajectory, inspect its derivatives, and explain the observed motion

1. Choose q0 and q1 for a six-axis robot.
2. Generate a joint trajectory with jtraj.
3. Plot q, qd and qdd. Identify the endpoint conditions.
4. Generate a Cartesian trajectory between T0 and T1.
5. Solve IK along the Cartesian trajectory and compare the joint motion.
6. Repeat one case with tpoly and lspb. Explain the difference in velocity.

PART 4

Verification and application

Check the whole motion before it reaches the robot

1 Feasibility and verification criteria
2 Wheel-insertion worked example
3 Exercise, Python toolbox and summary

Feasibility Along the Whole Motion

CORE

A path is acceptable only when every configuration on it is acceptable

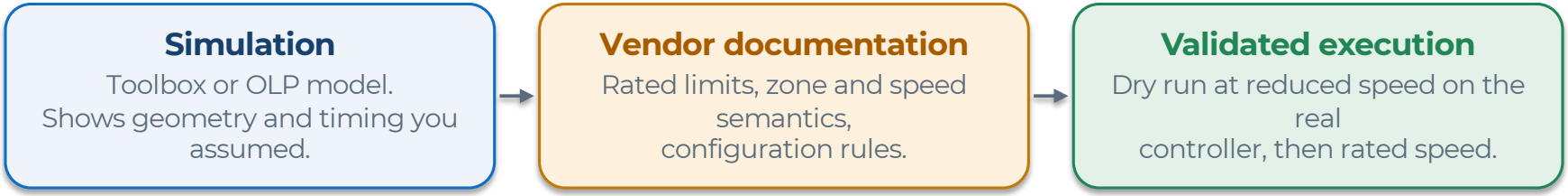
Check	Why endpoints are not enough
Joint limits	an intermediate pose can exceed a limit that both endpoints respect
Singularity proximity	joint speed can spike mid-path while the TCP speed stays constant
Self and cell collision	the elbow and the rear of the arm sweep where the TCP never goes
Tool and carried part	a panel or wheel sweeps a far larger volume than the TCP
Speed and acceleration	limits bind hardest between targets, not at them
Process orientation	the approach direction must hold along the contact segment, not only at its end

Sample the motion, evaluate every check at every sample, and keep the evidence. This is what a tender defence is made of.

What Counts as Verified

CORE

Three different kinds of evidence, and they are not interchangeable



A simulation proves the motion agrees with the model you built, including its simplifications.

Vendor documentation defines what an instruction means on that controller.

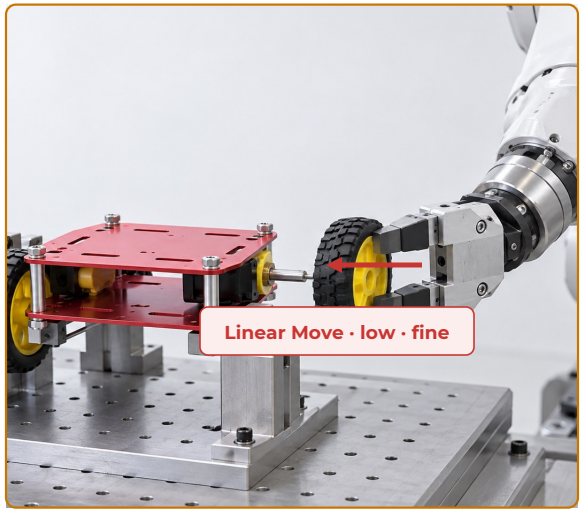
Only a supervised run on the machine proves the cell behaves as designed.

Say which kind of evidence you have. "Validated in simulation" and "validated on the robot" are different claims.

Example: A Motion Plan for Wheel Insertion

EXAMPLE

Selected targets only; the full report covers all eight motion stages



Target	Motion	Speed	Zone	Why
phome	PTP Move	rated	large	free space, no part carried
p_pickapp	PTP Move	rated	medium	above the wheel supply, clear of the fixture
ppick	Linear Move	reduced	fine	vertical approach to grip; must be settled
p_shaftapp	PTP Move	rated	medium	after pick retreat, transfer with the wheel
pinsert	Linear Move	low	fine	along the shaft axis; contact operation
pretreat	Linear Move	reduced	fine	straight back out before any transfer

Add pick retreat and final home to the complete eight-stage report; justify every target.

Peter Corke Robotics Toolbox for Python

OPTIONAL

Python provides the same trajectory concepts with different names and return objects

```
import roboticstoolbox as rtb
s  = rtb.quintic(0, 1, 51)
sb = rtb.trapezoidal(0, 1, 51)
tj = rtb.jtraj(q0, q1, 51)
tm = rtb.mtraj(rtb.quintic, q0, q1, 51)
tms = rtb.mstraj(via, dt, tacc, qdmax=qdmax, q0=q0)
Ts = rtb.ctraj(T0, T1, 51)
```

MATLAB tpoly corresponds to Python quintic. MATLAB lspb corresponds to Python trapezoidal. Inspect q, qd and qdd in the returned Trajectory object.

Trajectory Planning Summary

CORE

Concepts, toolbox details and project decisions form one workflow

Concept level
Choose the path, motion space and
time law before writing code.

→

MATLAB and Toolbox detail
Generate samples and inspect q, qd and qdd.
Keep one continuous IK branch.

→

Project application
Define targets, motion type, speed and zone
for each operation.

Trajectory planning turns a required geometric route into sampled joint references with an appropriate timing profile.

MATLAB makes the plan visible and checks limits, speed, acceleration and singularity proximity.

The project requires whole-motion checks, collision evidence and a clear reason for every target.

> The project deliverable combines the motion plan, engineering justification and verification evidence. A MATLAB plot supports this deliverable.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!